

BIOE 860 — Week 3

Project Structure, Hygiene & Version Control with Git

From final_v2_REALfinal.csv → a reproducible biomedical research project

BIOE 860 • Week 3

Learning objectives

- Organize a biomedical data-science project using a clear, reusable directory structure.
- Use consistent file naming and environment documentation.
- Explain what Git, GitHub, GitHub Desktop, repositories, commits, branches, forks, remotes, and pull requests are.
- Use the essential Git command-line workflow: status → add → commit → push/pull.
- Explain staging, HEAD, stash, merge, rebase, and merge conflicts at an introductory level.
- Create a repository and practice a safe branch-based workflow.

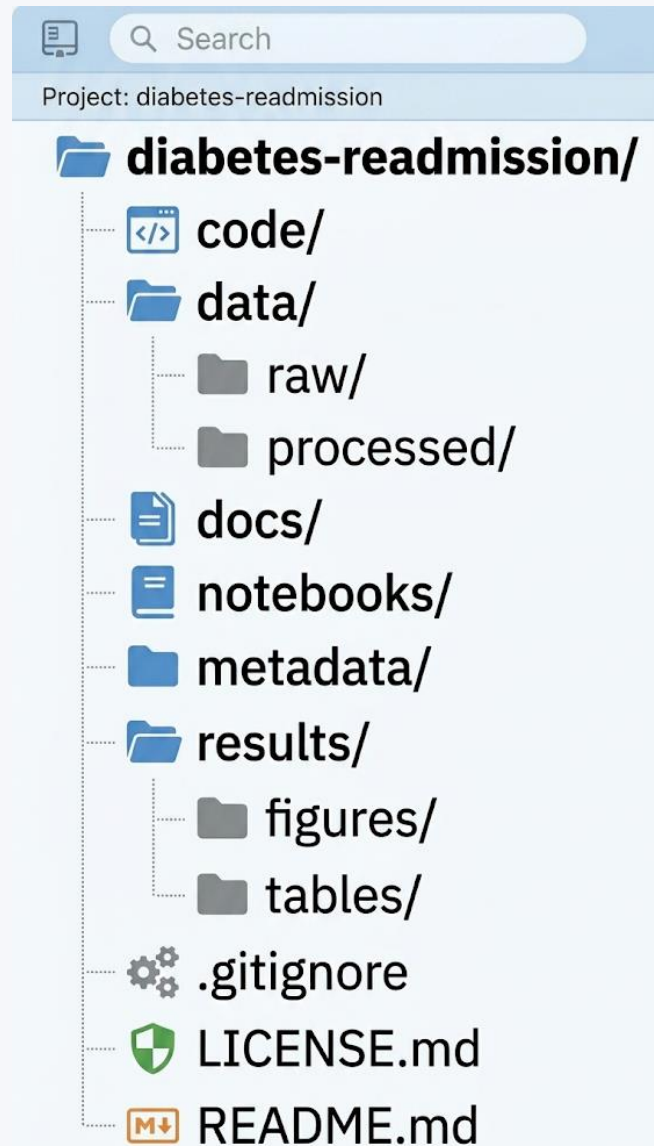
The problem: research projects decay without structure

- Files accumulate: data_new.csv, data_new2.csv, data_final.csv, data_final REALLY.csv.
- Scripts depend on undocumented paths and manual steps.
- Raw data get overwritten during cleaning.
- Figures cannot be traced back to the code and input data that produced them.
- Different collaborators have different versions of the same project.
- Six months later, even the original analyst may not remember what happened.

A simple project structure

```
diabetes-readmission/  
|— code/  
|— data/  
|   |— raw/  
|   |__ processed/  
|— docs/  
|— notebooks/  
|— metadata/  
|— results/  
|   |— figures/  
|   |__ tables/  
|— .gitignore  
|— LICENSE.md  
|__ README.md
```

A simple project structure



Raw data are read-only

- raw/ contains the data exactly as received or acquired.
- Never manually edit the only copy of raw data.
- Cleaning scripts read raw data and write new files to processed/.
- Processed data should be reproducible from raw data + code.
- If raw data are sensitive or too large, they may not belong in Git at all.
- Document where protected data live and how authorized users obtain them.

File naming: boring is good

- Prefer names that are meaningful, sortable, and machine-friendly.
- Use dates consistently: YYYY-MM-DD sorts naturally.
- Avoid spaces and ambiguous labels such as final, latest, new, or temp.
- Use lowercase and separators consistently, e.g. snake_case.
- Let Git track versions instead of encoding version history into filenames.

GOOD

2026-08-26_cohort_definition.md
clean_labs.R
figure_01_readmission_rate.png

AVOID

Final Data NEW 2.csv
analysis_latest_REALFINAL.R
Figure good version.png

What is version control?

- A system that records changes to files over time.
- Allows you to inspect what changed, when, and by whom.
- Supports recovery from mistakes.
- Enables collaboration without emailing ZIP files back and forth.
- Allows multiple lines of work to coexist.

```
main
|
A --- B --- C
      \
       D --- E
           new-model
```


Git, GitHub, and GitHub Desktop are different things

Git — Version-control software that runs locally on your computer.

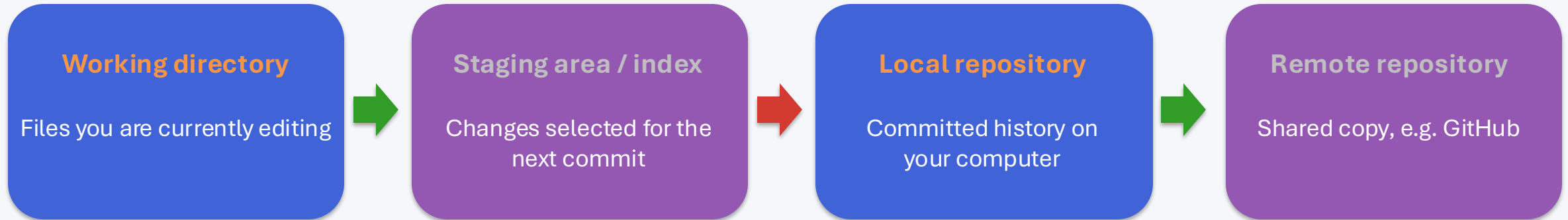
GitHub — A web platform that hosts Git repositories and adds collaboration tools.

GitHub Desktop — A graphical application that lets you perform many Git/GitHub operations without typing commands.

Why learn the command line?

- **Universal:** the same Git commands work across editors and many computing environments.
- **Transparent:** you see exactly which operation you are asking Git to perform.
- **Reproducible:** commands can be documented in tutorials and scripts.
- **Remote-friendly:** essential when working on servers, clusters, or cloud systems without a desktop GUI.
- **Powerful:** advanced operations are often easier to understand and troubleshoot from the command line.
- **GitHub Desktop is still useful:** a GUI can make diffs, commits, and branch switching more approachable.

The Git mental model: four places



edit → git add → git commit → git push

git pull brings remote changes back toward your local working copy.

Your first local repository

```
mkdir diabetes-readmission  
cd diabetes-readmission  
  
git init  
git status  
  
git add README.md  
git commit -m "Add project README"  
  
git log --oneline
```

Essential commands: inspect before you change

```
git status          # What changed?  
git diff            # Unstaged line-by-line changes  
git diff --staged   # Changes selected for next commit  
git log --oneline    # Compact commit history  
git show <commit>   # Inspect a commit
```

Habit: run `git status` frequently. It is one of the safest commands in Git.

Staging: choose what belongs in the next commit

- The staging area is a preparation area for the next snapshot.
- You do not have to commit every changed file together.
- A good commit represents one coherent idea.
- Use `git add <file>` to stage changes; `git restore --staged <file>` to unstage them.

```
git add code/clean_labs.R
git add metadata/data_dictionary.csv

git status

git commit -m "Standardize laboratory units"
```

What is HEAD?

- HEAD is Git's reference to your currently checked-out position.
- Usually, HEAD points to the current branch, which points to the latest commit on that branch.
- When you commit, the branch pointer moves forward to the new commit.
- Understanding HEAD helps make commands such as `git diff HEAD` and `git reset less` mysterious.

```
A --- B --- C
      ^
    main
      ^
    HEAD
```

The diagram illustrates the relationship between HEAD, a branch, and commits. At the top, three commits are shown in a sequence: A, B, and C, connected by dashed lines. Below commit B, an upward-pointing caret (^) indicates that the 'main' branch points to commit B. Below the 'main' branch name, another upward-pointing caret (^) indicates that the 'HEAD' pointer points to the 'main' branch.

What is a branch?

- A branch is a movable pointer to a line of commits.
- It lets you develop a feature, analysis, or experiment without destabilizing main.
- Branches are lightweight and cheap: create them often for meaningful work.
- Examples: feature/new-model, analysis/sensitivity-check, fix/lab-units.

```
A --- B --- C main
      \
      D --- E analysis/new-model
```


Branch workflow

```
git switch -c analysis/new-model  
  
# edit files  
git status  
git add code/model.R  
git commit -m "Add baseline readmission model"  
  
git switch main  
git merge analysis/new-model
```

What is a remote?

- A remote is a named reference to another Git repository.
- origin is the conventional default name for the repository you cloned from or connected to.
- git push sends local commits to a remote.
- git fetch downloads remote history without integrating it.
- git pull usually fetches and then integrates remote changes.

```
git remote -v  
git push -u origin main  
git fetch origin  
git pull
```

GitHub: collaboration around Git

- Host and share repositories.
- Track issues and discussions.
- Review changes with pull requests.
- Protect important branches and require review.
- Automate checks or workflows with GitHub Actions.
- GitHub is not required to use Git—but it is a common collaboration platform.

What is a pull request?

- A pull request (PR) proposes that changes from one branch be reviewed and integrated into another.
- It creates a place to inspect diffs, discuss changes, run checks, and request revisions.
- A pull request is not itself a Git commit and is not the same thing as a merge.
- Typical workflow: branch → commits → push → pull request → review → merge.

What is a fork?

- A fork is your own server-side copy of someone else's repository, typically on GitHub.
- Use a fork when you do not have write access to the original repository or want an independent copy.
- Common open-source workflow: fork → clone → branch → commit → push → pull request to upstream.
- For a small internal team where everyone has write access, ordinary branches are often simpler than forks.

Branch or fork? A simple decision guide

Situation	Usually use	Why
Solo experiment inside your repo	Branch	Isolate work without copying the repository
Team member with write access	Branch + PR	Shared repository, review before integration
Contributing to another lab/open-source repo	Fork + branch + PR	You may not have write access
Long-term independent derivative project	Fork	Maintain a separate repository history

Merge: combine histories

Before merge:

```
A --- B --- C main
      \
      D --- E feature
```

After a merge commit:

```
A --- B --- C ----- M main
      \       /
      D --- E -----
```

Rebase: replay commits on a new base

Before:

A --- B --- C main

\

D --- E feature

After rebase of feature onto main:

A --- B --- C --- D' --- E' feature

- Rebase can create a cleaner linear history.
- It rewrites commit identities/history.
- Beginner rule: avoid rebasing commits that other people are already using unless your team explicitly agrees.

Merge vs. rebase

	Merge	Rebase
Core idea	Combine histories	Replay commits on a new base
History	Preserves branch structure	Can make history linear
Risk	Usually straightforward	Rewrites commits
Beginner default	Safe choice for shared work	Use deliberately; avoid rewriting shared history

What is a merge conflict?

- A conflict occurs when Git cannot safely decide how to combine competing changes.
- Typical example: two branches edit the same lines differently.
- A conflict is not corruption. Git pauses and asks a human to choose the intended result.
- Resolve the file, remove conflict markers, test the result, stage it, and complete the merge/rebase.

```
<<<<<< HEAD
threshold <- 0.50
=====
threshold <- 0.65
>>>>>> analysis/new-model
```

Conflict resolution: a calm workflow

1. Read Git's message and run `git status`.
2. Open each conflicted file.
3. Decide what the final content should be; do not merely delete markers blindly.
4. Remove `<<<<<<`, `=====`, and `>>>>>>` markers.
5. Run/test the code if possible.
6. `git add` the resolved file.
7. Complete the merge with `git commit`, or continue a rebase with `git rebase --continue`.
8. If necessary, abort safely: `git merge --abort` or `git rebase --abort`.

What is git stash?

- git stash temporarily shelves uncommitted changes.
- Useful when you need a clean working directory before switching context.
- git stash list shows saved stashes.
- git stash pop reapplies the most recent stash and removes it from the stash list.
- Stash is temporary storage—not a substitute for meaningful commits.

```
git status
git stash
git switch main
...
git switch analysis/new-model
git stash pop
```

What should NOT go into Git?

- Protected patient-level clinical data.
- Passwords, API keys, tokens, or credentials.
- Large generated files that can be recreated.
- Temporary files, caches, editor artifacts.
- Environment-specific files that do not belong in the shared project.

```
.gitignore  
  
data/raw/*  
.env  
.Rhistory  
.Rproj.user/  
__pycache__/  
*.tmp  
results/cache/
```

README: the front door to the project

- Project purpose and research question.
- Directory structure.
- How to obtain/access data.
- Software/environment requirements.
- How to reproduce the analysis.
- Important governance or access restrictions.
- Contact/ownership information when appropriate.

Document the computing environment

- Record language and package versions.
- For R: consider `renv` and `sessionInfo()`.
- For Python: `requirements.txt`, `pyproject.toml`, `conda environment.yml`, or similar.
- Do not assume your future computer will have today's package versions.
- Environment capture + Git history + documented data provenance greatly improves reproducibility.

A safe everyday Git workflow

```
git pull

git switch -c analysis/readmission-model

# work...
git status
git diff

git add code/model.R
git commit -m "Add baseline logistic model"

git push -u origin analysis/readmission-model

# Open pull request → review → merge
```


Lab: build a reproducible BIOE 860 repository

1. Create the provided starter project structure.
2. Initialize Git and inspect git status.
3. Create README.md and .gitignore.
4. Make the first commit.
5. Create a branch named analysis/first-summary.
6. Add a small analysis script and commit it.
7. Merge the branch into main.
8. Create and resolve a deliberate merge conflict.
9. Connect/publish the repository to GitHub using GitHub Desktop or command line.
10. Reflection: identify which project files should never be pushed to GitHub.

GitHub Desktop lab path

1. File → Add/Create repository.
2. Review the Changes panel before committing.
3. Write a meaningful summary and commit to the current branch.
4. Create/switch branches from the branch menu.
5. Publish repository / push origin.
6. Fetch/pull before beginning new work.
7. Use the History tab to inspect commits and diffs.

Command-line lab path

```
git init
git status
git add .
git commit -m "Initialize BIOE 860 project"
```

```
git switch -c analysis/first-summary
git add code/summary.R
git commit -m "Add first summary analysis"
```

```
git switch main
git merge analysis/first-summary
```

Week 3 take-home messages

- Project organization is part of reproducible science.
- Keep raw data immutable and transformations scripted.
- Let Git track versions instead of inventing filenames such as final_final2.
- Understand the mental model: working tree → staging area → local commits → remote.
- Branches isolate work; forks create independent server-side copies; pull requests support review.
- Merge combines histories; rebase rewrites/replays commits and should be used deliberately.
- Conflicts are normal and resolvable.
- Never commit protected data or secrets.

Quick retrieval check

What is the difference between Git and GitHub?

What does the staging area do?

When would you use a branch instead of a fork?

What is the difference between merge and rebase?

What does HEAD refer to?

What should you do when Git reports a conflict?

What belongs in .gitignore?

Why should raw biomedical data remain unchanged?